

CONTINUOUS ASSESSMENT TEST – II
Regulations R 2023 – V1.1

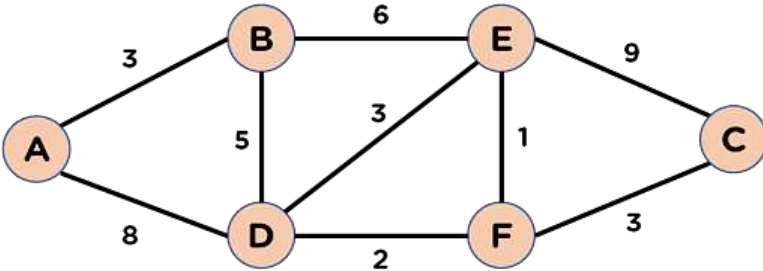
Department of Computer Science Engineering
Second Year / Third Semester

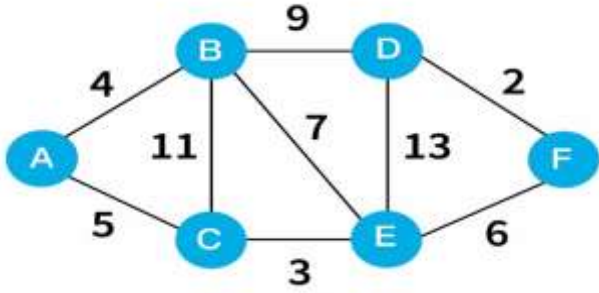
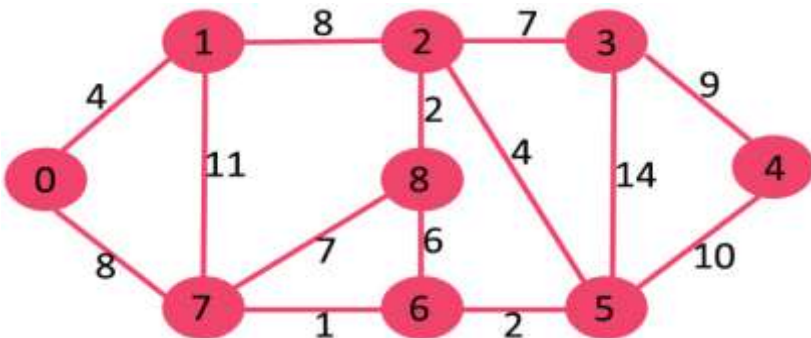
2311CSC302J – Data Structures and Algorithms
(CSE, AIML, AI&DS, CSE (CS), CSBS, CSD)

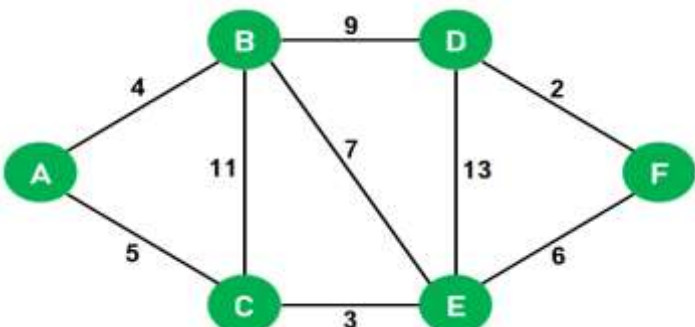
CO1: Infer the concepts of abstract data types for linear data structures.
CO2 Apply non-linear data structures concepts in real time applications.
CO3 Develop solutions using linear and non- linear data structures.
CO4 Illustrate various searching and sorting techniques to solve the complex problem.
CO5 Evaluate space and time complexity for a given algorithm.

Unit – III NON-LINEAR DATA STRUCTURES - GRAPHS

Q. No	Questions	COs	Blooms Level
PART A			
1.	List any two applications of minimum spanning trees.	CO3	K1
2.	What is the role of edge weights in MST algorithms?	CO3	K2
3.	What is the initial step in Dijkstra's algorithm?	CO3	K1
4.	How does a priority queue help in Dijkstra's algorithm?	CO3	K2
5.	Describe the difference in the approach between Kruskal's and Dijkstra's algorithm.	CO3	K2
6.	Describe the purpose of using minimum spanning tree algorithms in network design.	CO3	K2
7.	Name any two algorithms used for finding a minimum spanning tree.	CO3	K1
8.	How does Kruskal's algorithm ensure no cycles are formed in the MST?	CO3	K2
9.	Differentiate between tree and spanning tree.	CO3	K2
10.	What is the basic requirement for applying Kruskal's algorithm?	CO3	K1

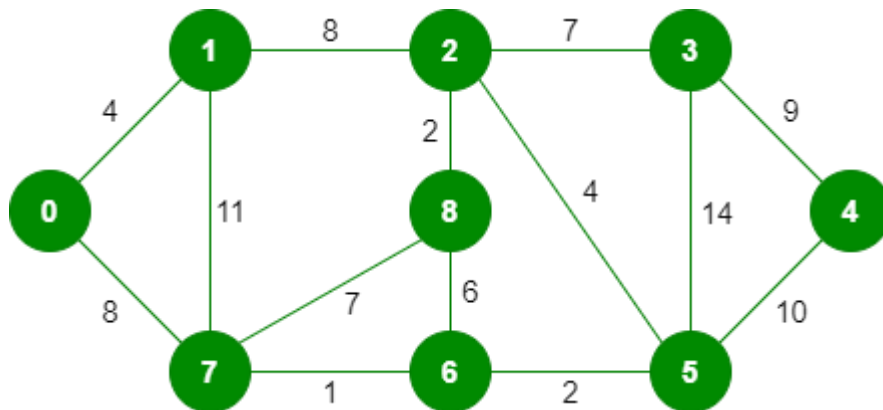
Q. No	Questions	COs	Blooms Level
Part B			
1.	Justify the use of Kruskal's Algorithm for finding the Minimum Spanning Tree in a sparse weighted graph. Solve the following graph and explain your reasoning. 	CO3	K3

2.	Apply Dijkstra's algorithm to the given graph and determine the shortest path from source vertex A. Clearly show the table used for updating distances. 	CO3	K3
3.	Evaluate the impact of choosing different starting vertices in Kruskal's algorithm. Does it affect the final MST? Support your answer with reasons.	CO3	K3
4.	Analyze the working of Dijkstra's algorithm for finding the shortest path in a weighted graph with an example.	CO3	K4
5.	Evaluate the shortest path from a source vertex to all other vertices in the following graph using Dijkstra's Algorithm. 	CO3	K5

Q.No	Questions	COs	Blooms Level
Part C			
1.	Write down the procedure of Dijkstra's algorithm to find the shortest path from the source node A. 	CO3	K5

2.

Create a minimum spanning tree using Kruskal's algorithm for the following weighted graph. Show all steps including edge selection and cycle checking.



CO3

K6

Unit – IV SORTING, SEARCHING AND HASH TECHNIQUES

Q. No	Questions	COs	Blooms Level
Part A			
1.	Classify the different sorting methods.	CO4	K1
2.	Distinguish between linear and binary search technique.	CO4	K1
3.	State the worst-case time complexity of Bubble Sort.	CO4	K2
4.	What is the pivot in Quick Sort and why is it important?	CO4	K1
5.	How is Merge Sort different from Quick Sort in terms of space complexity?	CO4	K2
6.	What is the time complexity of Binary Search in the worst case?	CO4	K2
7.	What is separate chaining in hashing?	CO4	K1
8.	Define extendible hashing.	CO4	K1
9.	Mention one drawback of open addressing.	CO4	K2
10.	How does open addressing differ from separate chaining?	CO4	K2
11.	Why is Binary Search more efficient than Linear Search for large datasets?	CO4	K2
12.	When can Binary Search be used?	CO4	K2
13.	How does extendible hashing use directory and buckets?	CO4	K2
14.	How many recursive calls are made in Merge Sort for an array of 8 elements?	CO4	K2
15.	What is a collision in hashing?	CO4	K1

Q. No	Questions	COs	Blooms Level
Part B			
1.	Interpret an algorithm to sort a set of 'N' numbers using bubble sort and demonstrate the sorting steps for the following set of numbers: 30,52,29,87,63,27,19,54	CO4	K5
2.	Write a C program to search a number 89 from the given list 20, 35,46,57,67,79,89,92 using binary search.	CO4	K4
3.	Given the same key sequence and table size, insert the keys using: (a) Separate Chaining (b) Linear Probing Keys: 21, 31, 41, 51, 61 (Table size = 10), Compare the final structure of the hash table for both methods.	CO4	K5
4.	Apply the Quick Sort algorithm to sort the list: [45, 20, 35, 70, 10, 90, 15]. Use the first element as the pivot. Show all steps including partitioning and recursive calls.	CO4	K3
5.	Evaluate the effectiveness of Extendible Hashing in handling dynamic insertions using the following keys: Keys: 3, 7, 12, 14, 24, 30 Assume: Bucket size = 2 Initial global depth = 1	CO4	K5

	Use the last few bits of binary representation for hashing		
6.	Evaluate the pros and cons of using Merge Sort over other sorting algorithms for large datasets stored in external memory (e.g., files).	CO4	K4
7	A teacher wants to sort the test scores of 5 students: [88, 75, 90, 70, 85] so she can quickly identify the highest and lowest performers. She uses the Bubble Sort algorithm to perform the sort. Analyze how Bubble Sort works on this data — explain how the algorithm progresses through each pass.	CO4	K4
8	Binary Search is more efficient than Linear Search but has limitations. Analyze these limitations and give examples where Binary Search fails.	CO4	K4

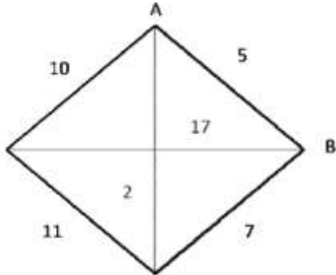
Q. No	Questions	COs	Blooms Level
Part C			
1.	Insert the following keys into a hash table of size 10 using the hash function $h(k) = k \% 10$. Use separate chaining to handle collisions. Keys: 25, 35, 15, 20, 30, 50.	CO4	K6
2.	An e-commerce company receives a large volume of daily transactions that are stored in an unsorted array:[1250, 4300, 2999, 1800, 5150, 3200, 750, 6100, 4600, 2700].As a software engineer, you are asked to design and implement an efficient sorting strategy to organize these transactions in ascending order using the Merge Sort algorithm.	CO4	K6

Unit – V ALGORITHM DESIGN TECHNIQUES

Q. No	Questions	COs	Blooms Level
Part A			
1.	Define backtracking.	CO5	K1
2.	What is the goal of the N-Queen problem?	CO5	K1
3.	Explain how backtracking is used in solving the N-Queen problem.	CO5	K2
4.	Why is backtracking considered a depth-first search technique?	CO5	K2
5.	List two problems that can be solved using backtracking.	CO5	K1
6.	Define Branch and Bound.	CO5	K1
7.	What type of problem is Travelling Salesman Problem (TSP)?	CO5	K1
8.	How does bounding help in Branch and Bound method?	CO5	K2
9.	Why is Branch and Bound suitable for TSP?	CO5	K2
10.	Mention two key components of the Branch and Bound algorithm.	CO5	K1
11.	What is the principle of optimality in Dynamic Programming?	CO5	K1
12.	State the difference between 0/1 Knapsack and Fractional Knapsack.	CO5	K1
13.	How does Dynamic Programming solve the 0/1 Knapsack problem?	CO5	K2
14.	Why is Dynamic Programming preferred over recursion for knapsack problems?	CO5	K2
15.	What strategy does the greedy method follow in Fractional Knapsack?	CO5	K1

Q. No	Questions	COs	Blooms Level												
Part B															
1.	Apply the backtracking technique to solve the 4-Queen problem. Draw the state space tree and explain each step in placing the queens.	CO5	K3												
2.	Analyze the principles of Dynamic Programming and explain how it differs from Divide and Conquer.	CO5	K4												
3.	Analyze the differences between 0/1 Knapsack and Fractional Knapsack. For a knapsack of capacity 50 and the following items: <table><tr><td>Item</td><td>Profit</td><td>Weight</td></tr><tr><td>1</td><td>60</td><td>10</td></tr><tr><td>2</td><td>100</td><td>20</td></tr><tr><td>3</td><td>120</td><td>30</td></tr></table> Explain which technique (DP or Greedy) is suitable and why.	Item	Profit	Weight	1	60	10	2	100	20	3	120	30	CO5	K4
Item	Profit	Weight													
1	60	10													
2	100	20													
3	120	30													
4.	Evaluate how Dynamic Programming improves efficiency in solving the 0/1 Knapsack problem for the following: <table><tr><td>Items</td><td>Weight</td><td>Profit</td></tr><tr><td>1</td><td>18</td><td>54</td></tr><tr><td>2</td><td>19</td><td>38</td></tr><tr><td>3</td><td>15</td><td>60</td></tr></table> Compare the time complexity with recursion.	Items	Weight	Profit	1	18	54	2	19	38	3	15	60	CO5	K5
Items	Weight	Profit													
1	18	54													
2	19	38													
3	15	60													
5.	Apply the Greedy method to solve the Fractional Knapsack problem:	CO5	K3												

	W=6, <table><tr><th>Items</th><th>Weight</th><th>Value</th></tr><tr><td>1</td><td>2</td><td>8</td></tr><tr><td>2</td><td>3</td><td>6</td></tr><tr><td>3</td><td>2</td><td>4</td></tr></table> Show step-by-step selection based on profit/weight ratio.	Items	Weight	Value	1	2	8	2	3	6	3	2	4		
Items	Weight	Value													
1	2	8													
2	3	6													
3	2	4													
6.	Evaluate why Greedy method fails for 0/1 Knapsack problem using the following items:Capacity = 50, Items = (Profit, Weight): (60,10), 100,20), (120,30),Compare the Greedy result with the DP result.	CO5	K5												
7.	Analyze how the Backtracking algorithm solves the 4-Queens problem. Illustrate the recursive function with decision points and show how backtracking eliminates invalid solutions.	CO5	K4												
8.	Analyze the process of solving the Travelling Salesperson Problem (TSP) using the Branch and Bound method.Explain how the cost matrix is used, how lower bounds are calculated, and how unnecessary paths are eliminated during the search for the optimal solution.	CO5	K4												

Q. No	Questions	COs	Blooms Level															
Part C																		
1.	Plan the following instance of the 0/1, knapsack problem given the knapsack capacity in W=10 using dynamic programming and explain it. Item Profit Weight <table><tr><th>Item</th><th>Profit</th><th>Weight</th></tr><tr><td>1</td><td>10</td><td>5</td></tr><tr><td>2</td><td>40</td><td>4</td></tr><tr><td>3</td><td>30</td><td>6</td></tr><tr><td>4</td><td>50</td><td>3</td></tr></table>	Item	Profit	Weight	1	10	5	2	40	4	3	30	6	4	50	3	CO5	K5
Item	Profit	Weight																
1	10	5																
2	40	4																
3	30	6																
4	50	3																
2.	Evaluate the performance of Branch and Bound in solving the Travelling Salesman Problem (TSP) for the following cost matrix:  Show the bounding process and pruning decisions.	CO5	K5															